树链剖分（Heavy-Light Decomposition）

1. 算法概述

树链剖分是一种将树形结构转化为线性序列的算法，通过对树进行特殊的剖分，使得树上路径和子树操作能够高效处理。其核心思想是将树分解为若干条“重链”，将树上的操作转化为对 $O(\log n)$ 条链的操作。

具体来说，将整棵树剖分为若干条链，使它组合成线性结构，然后用其他的数据结构维护信息。

树链剖分（树剖/链剖）有多种形式，如重链剖分、长链剖分和用于 Link/cut Tree 的剖分（有时被称作「实链剖分」）。大多数情况下（没有特别说明时），「树链剖分」都指「重链剖分」。

重链剖分可以将树上的任意一条路径划分成不超过 $O(\log n)$ 条连续的链，每条链上的点深度互不相同（即是自底向上的一条链，链上所有点的 LCA 为链的一个端点）。

重链剖分还能保证划分出的每条链上的结点 DFS 序连续，因此可以方便地用一些维护序列的数据结构（如线段树）来维护树上路径的信息。例如：

- 修改树上两点之间的路径上所有点的值。
- 查询树上两点之间的路径上结点权值的和/极值/其它（在序列上可以用数据结构维护，便于合并的信息）。

除了配合数据结构来维护树上路径信息，树剖还可以用来 $O(\log n)$（且常数较小）地求 LCA。在某些题目中，还可以利用其性质来灵活地运用树剖。

1.1 主要应用

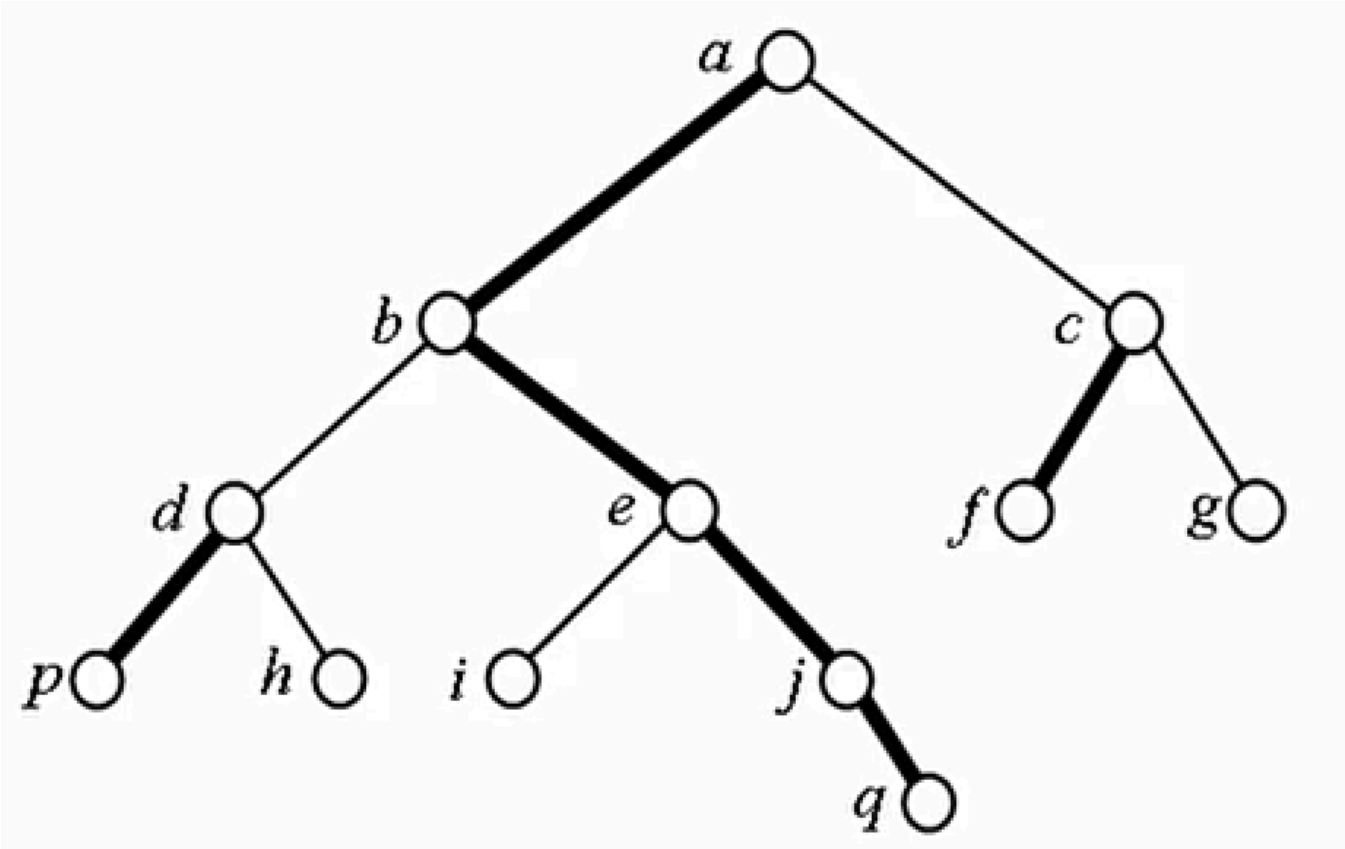
- **路径查询/修改**：求树上两点间路径的和、最值等
- **子树查询/修改**：求子树的和、最值等
- **最近公共祖先（LCA）**：快速查询两节点的最近公共祖先
- **树上动态维护**：结合线段树等数据结构实现动态操作

1.2 算法优势

- 将树上路径操作转化为 $O(\log n)$ 个区间操作
- 配合线段树等数据结构，实现 $O(\log_2 n)$ 的路径操作复杂度
- 代码结构清晰，预处理后操作简洁高效

2. 核心概念与定义

2.1 基本概念

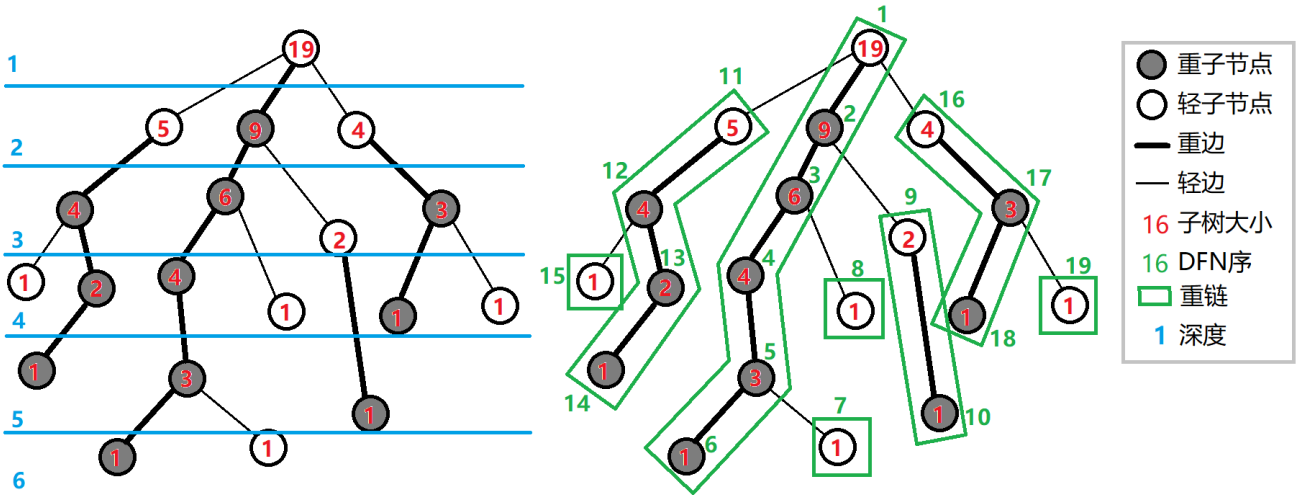


对于有根树（设根为 `root`）中的每个节点 `u`：

概念	定义	说明
重儿子 `son[u]`	子树大小最大的儿子节点	如有多个儿子子树大小相同，任选一个
轻儿子	除重儿子外的其他儿子节点	
重边	连接节点与其重儿子的边	

概念	定义	说明
轻边	连接节点与其轻儿子的边	
重链	由重边连续连接形成的链	每条链以轻儿子或根节点开始
链头 `top[u]`	节点所在重链的顶端节点	即一条重链上深度最小的点

2.2 重链剖分的性质



- 树上每个结点都属于且仅属于一条重链。
- 重链开头的结点不一定是重子结点（因为重边是对于每一个结点都有定义的）。
- 所有的重链将整棵树完全剖分。
- 在剖分时重边优先遍历，最后树的 DFS 序上，重链内的 DFS 序是连续的。按 DFN 排序后的序列即为剖分后的链。(把这段连续的数字看成"线段",线段内的区间问题用线段树来处理正合适)
- 一棵子树内的 DFS 序是连续的。
- 从任意一个点出发，到根节点的路径上经过的重链不会超过 $O(\log_2 n)$ 条。由于每两条重链之间是一条轻边，这个性质也等价于：经过的轻边不会超过$O(\log_2 n)$条

下面证明：经过的轻边不会超过$O(\log_2 n)$条，以二叉树为例，x的一个轻儿子y，y的子树大小必然小于x的子树大小的一半。从根节点向任意节点走，设size为当前节点的子树大小，那么每经过一条轻边，size至少除以2，所以，最后到达叶子节点时，最多经过了$O(\log_2 n)$条轻边。如果是多叉树，每经过一条轻边，size减少得更快，经过的轻边也少于$O(\log_2 n)$条。

2.3 关键数组说明

以洛谷P3379【模板】最近公共祖先（LCA）为例给出树链剖分求LCA的示例。

预处理过程中需要维护以下数组：

```
const int N = 500005;  // 根据题目要求调整大小
int fa[N];     // 父节点
int deep[N];   // 节点深度（根深度为0或1）
int siz[N];    // 以u为根的子树大小
int son[N];    // 重儿子编号
int top[N];    // 节点所在重链的链头
```

3. 预处理：两遍DFS

3.1 数据结构：链式前向星

```
#include<bits/stdc++.h>
using namespace std;

const int N = 500005;

struct Edge {
    int to, next;
} edge[2*N];  // 无向图需要两倍空间

int head[2*N], cnt;

void init() {
    for(int i = 0; i < 2*N; ++i) {
        edge[i].next = -1;
        head[i] = -1;
    }
    cnt = 0;
}

void addedge(int u, int v) {
    edge[cnt].to = v;
    edge[cnt].next = head[u];
    head[u] = cnt++;
}
```

3.2 第一遍DFS：计算基本信息

```cpp
int deep[N], siz[N], son[N], fa[N];

void dfs1(int x, int father) {
    // 初始化节点信息
    deep[x] = deep[father] + 1;
    fa[x] = father;
    siz[x] = 1;

    // 遍历所有子节点
    for(int i = head[x]; ~i; i = edge[i].next) {
        int y = edge[i].to;
        if(y == father) continue;

        // 递归处理子节点
        dfs1(y, x);

        // 更新子树大小
        siz[x] += siz[y];

        // 更新重儿子
        if(!son[x] || siz[y] > siz[son[x]]) {
            son[x] = y;
        }
    }
}
```

第一次DFS的功能:

1. 计算每个节点的深度 `deep[x]`
2. 记录每个节点的父节点 `fa[x]`
3. 计算每个节点的子树大小 `siz[x]`
4. 确定每个节点的重儿子 `son[x]`

3.3 第二遍DFS：进行重链剖分

```
int top[N];  // 链头数组

void dfs2(int x, int topx) {
    // 记录当前节点所在链的链头
    top[x] = topx;

    // 如果没有重儿子，说明是叶子节点
    if(!son[x]) return;

    // 优先处理重儿子，保持重链连续
    dfs2(son[x], topx);

    // 处理轻儿子
    for(int i = head[x]; ~i; i = edge[i].next) {
        int y = edge[i].to;
        // 跳过父节点和重儿子
        if(y != fa[x] && y != son[x]) {
            // 每个轻儿子都开始一条新的重链
            dfs2(y, y);
        }
    }
}
```

第二次DFS的功能：

1. 为每个节点分配链头 `top[x]`
2. 保证每条重链上的节点有连续的DFS序
3. 完成重链剖分

4. 基于树链剖分的LCA查询

4.1 LCA查询实现

```
int LCA(int x, int y) {
    // 当两个节点不在同一重链时，持续向上跳
    while(top[x] != top[y]) {
        // 选择链头更深的节点向上跳
        if(deep[top[x]] < deep[top[y]]) {
            swap(x, y);
        }
        // 跳到当前链头的父节点
        x = fa[top[x]];
    }

    // 现在x和y在同一重链上
    // 深度较小的节点就是LCA
    return deep[x] < deep[y] ? x : y;
}
```

4.2 LCA查询原理解析

1. **比较链头**：检查两个节点是否在同一重链
2. **重链跳跃**：如果不在同一重链，将链头较深的节点跳到其链头的父节点
3. **重复过程**：直到两个节点在同一重链
4. **确定LCA**：同一重链上深度较小的节点即为LCA

4.3 时间复杂度分析

- 每次跳跃至少跨越一条轻边
- 从任意节点到根节点最多经过 $O(\log n)$ 条轻边
- LCA查询的时间复杂度：$O(\log n)$

5. 主函数与完整调用示例

```
int main() {
    // 初始化链式前向星
    init();

    int n, m, root;
    scanf("%d%d%d", &n, &m, &root);

    // 读入树结构
    for(int i = 1; i < n; i++) {
        int u, v;
        scanf("%d%d", &u, &v);
        addedge(u, v);
        addedge(v, u);  // 无向图添加双向边
    }

    // 树链剖分预处理
    dfs1(root, 0);   // 第一次DFS，计算基本信息
    dfs2(root, root); // 第二次DFS，进行重链剖分

    // 处理LCA查询
    while(m--) {
        int a, b;
        scanf("%d%d", &a, &b);
        printf("%d\n", LCA(a, b));
    }

    return 0;
}
```

6. 树链剖分的典型应用

- 修改点x到点y的路径上各点的权值。
- 查询点x到点y的路径上结点权值之和。
- 修改点x子树上各点的权值。
- 查询点x子树上所有结点的权值之和

TODO: 重链剖分/树链剖分

7. 算法性能分析

7.1 时间复杂度

操作	预处理	单次查询/修改	说明
LCA查询	$O(n)$	$O(\log n)$	基于重链跳跃
路径查询	$O(n)$	$O(\log n)$	拆分$O(\log n)$条重链
路径修改	$O(n)$	$O(\log n)$	每段重链$O(\log n)$处理
子树查询	$O(n)$	$O(\log n)$	DFS序连续区间
子树修改	$O(n)$	$O(\log n)$	DFS序连续区间

7.2 空间复杂度

- 存储树结构：$O(n)$
- 剖分数组：$O(n)$
- 线段树：$O(4n) = O(n)$

8. 总结

树链剖分是一种强大的树结构处理算法，它将树形结构转化为线性序列，使得许多区间操作数据结构能够应用于树上问题。通过重链剖分，实现了：

1. **高效的LCA查询**：$O(\log n)$ 时间复杂度
2. **灵活的路径操作**：支持查询和修改，$O(\log n)$ 时间复杂度
3. **简便的子树操作**：$O(\log n)$ 时间复杂度

虽然预处理稍复杂，但一次预处理后可以支持多种高效操作，是解决树上路径和子树问题的利器。掌握树链剖分的关键在于理解**两次DFS的预处理过程**和**路径跳跃的思想**。